

MixBytes()

[SECURITY AUDIT REPORT]

Lido Execution Delegation Framework

PREPARED FOR LIDO

REPORT DATE

September 11, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	8
1.5 Risk Classification	10
1.6 Summary of Findings	11
2. Findings Report	11
2.1 Critical	12
2.2 High	12
2.3 Medium	12
2.4 Low	13
L-1 DepositSecurityModule guardian set is never reconciled with DelegationContract termination or revocation; effective quorum shrinks silently and dead contracts pass addGuardian	13
L-2 DelegationContract.nominateDelegate(): contract delegates allow silent re-delegation of a seat to another controlling entity and self-delegation loops	14
L-3 DepositSecurityModule._addGuardian() / _isValidGuardianSignature(): any ERC-165-advertising account can be a guardian, and the ECDSA acceptance path LIP-37 says was removed is still compiled in	15
L-4 The accepted guardian signature length is defined only by a disagreement between two OpenZeppelin versions, is undocumented, and is not pinned by any test	16
L-5 depositBufferedEther and pauseDeposits docstrings state "each component taking 32 bytes", but the new guardian field is packed as 20 bytes. A signer following the docs produces invalid signatures	17
L-6 No admission path into the protocol checks a delegation contract's cooldown, and the contract does not bound it, so a zero-cooldown guardian or oracle member can be onboarded at any time	18
3. About MixBytes	19

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The Execution Delegation Framework separates the authority to hold a privileged role in Lido from the key that exercises it. `DelegationContract` is deployed through `DelegationFactory`; the contract's `OWNER` is immutable and identifies the entity controlling it, while a `delegate` hot key signs on its behalf and can be rotated by the owner without a DAO vote, subject to a per-contract `cooldown` that makes every nomination publicly visible before it takes effect. The contract answers ERC-1271, so any consumer can treat it as a signer, and the owner can revoke a delegate or irreversibly `terminate()` the seat. The engagement covered the framework itself together with its first consumer: `DepositSecurityModule v5`, redeployed so that every guardian is a `DelegationContract` rather than an EOA.

The interim security review was conducted over 4 working days by three auditors via manual code review and a proprietary AI-assisted analysis tool.

The review concentrated on what changes when a signature stops being a static ECDSA artefact and becomes a live, revocable contract call. Alongside that, we worked through our standard internal checklist covering access control, input validation, encoding and decoding correctness, arithmetic safety, reentrancy, and state-dependent logic. The project-specific vectors examined during this engagement are listed below.

Delegate state machine and rotation boundaries. We checked whether an old key can regain authority, whether two keys can be valid simultaneously, whether a key is usable before `activeFrom`, and whether `getDelegate()` can disagree with `isValidSignature()`. All four properties hold under the repository's fuzz and invariant tests and our own.

Signature verification across two OpenZeppelin versions. The DSM is pinned to OZ 4.4.1 and the `DelegationContract` to OZ 5.4.0, and the two libraries disagree on accepted signature lengths and on how a signer with code is verified. We examined the resulting acceptance surface, the retained `ecrecover` path in the DSM, and the ERC-165 gate on guardian registration.

An EIP-7702 delegation on a guardian's hot key disables the seat, and can stop deposits protocol-wide.

`DelegationContract.isValidSignature` forwards to OZ 5.4 `SignatureChecker`, which picks ECDSA or ERC-1271 by inspecting `delegate.code.length`. A hot-key EOA that signs a 7702 authorization - the ordinary "upgrade my wallet to a smart account" flow - permanently carries a 23-byte designator in its code slot, so the ERC-1271 branch is taken and the delegated implementation, which either lacks ERC-1271 or wraps the hash in an EIP-712/ERC-7739 envelope, does not recognise the bare digest the daemon signs. The seat returns `0xffffffff` while `getDelegate()` is unchanged and no event is emitted, so LIP-37's daemon startup check and the depositor-bot filter both pass. We recommend that the Key Handling Policy state that DSM and oracle

delegates must be plain EOAs; and, optionally, that `isValidSignature` treat a bare 7702 designator as an EOA so a wallet upgrade cannot change the seat's semantics.

A pending nomination cannot be cancelled without dropping the working delegate. The only function that clears `_pendingDelegate` is `revokeDelegate()`, which clears the current delegate too, and re-nominating the current key reverts with `AlreadyDelegate`. Re-nominating the correct address is safe only before `activeFrom`: past it, `_settle` promotes the wrong delegate first, so it holds full power for a whole cooldown before the same downtime is needed anyway. In the compromise case the cost is the same shape - if the current key is stolen one second before a pending key matures, the owner must revoke both and re-nominate, and the seat is dead for a full new cooldown even though the pending key had already been publicly visible for the entire reaction window. Both are owner-side mistakes or misfortune with bounded downtime rather than loss, but the one mechanism EDF adds for controlling entities - key rotation without governance - has no undo, and the workaround costs exactly the outage the mechanism is meant to avoid. We recommend an owner-only `cancelNomination()` that clears only the pending slot and emits a dedicated event, and letting `revokeDelegate()` promote a pending key whose visibility period has already elapsed.

The new DSM's `lastDepositBlock` starts at its deploy block. The constructor sets `lastDepositBlock = block.number`, and the migration does not carry the value over from the old DSM. After enactment the new DSM therefore reports the deployment block rather than the block of the protocol's actual last deposit. `canDeposit` takes the maximum of this value and the module's own `lastDepositToModuleBlock`, so the effect is at most a short additional wait before the first deposit and no state is lost. We note it only so that operators and monitoring reading `getLastDepositBlock()` after the vote are not misled.

The client acknowledges the guardian-compromise risk and relies on monitoring. A compromised guardian can already pause deposits and unvet every undeposited key in the protocol by calling DSM methods directly, so the client's position is that the more elaborate paths EDF opens up add nothing materially new to that baseline and do not warrant additional in-contract validation. We raised that this reasoning holds once a compromise is known, whereas an attacker who believes the leak is undetected has reason to stay quiet and abuse the new mechanisms instead, since their misuse is harder to notice than an outright halt; a seat can also be stalled through on-chain state by making `isValidSignature` revert. The client confirmed that depositor-bot transactions are monitored, that a failed transaction is the signal to `revokeDelegate()`, and that `terminate()` is used where the contract's key is suspected compromised. On that basis we agree the residual risk is acceptable, conditional on those failures remaining observable and being investigated. The corresponding findings are acknowledged and handled operationally.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Execution Delegation Framework
TYPE	Solidity
PLATFORM	EVM
TIMELINE	02.09.2026 - 10.09.2026

Scope of Audit

`src/DelegationContract.sol``src/DelegationFactory.sol``src/interfaces/IDelegationContract.sol``src/interfaces/IDelegationFactory.sol``contracts/0.8.9/DepositSecurityModule.sol`

Versions Log

Execution Delegation Framework

DATE	COMMIT HASH	NOTE
02.09.2026	557299104ad3eb1a74198933bd016328c490e276	Initial Commit

Lido Core

DATE	COMMIT HASH	NOTE
02.09.2026	8c4cee2be76a4ed8067026e30b5bdc64a4e3cb29	Initial Commit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
DelegationFactory.sol	0xD990770eB2B4b6062EDdB06892fF179C693b46e6	Ethereum
DepositSecurityModule.sol	0x39BB5d491e98A44D1bfe8047A737a81E296a63E0	Ethereum
LidoLocator.sol (impl)	0x60E09F1791F1168d0450E4F100616B4a3F95119C	Ethereum

We verified both contracts against the audited commits `8c4cee2be76a4ed8067026e30b5bdc64a4e3cb29` (Lido Core) and `557299104ad3eb1a74198933bd016328c490e276` (EDF), and they match. Both were created by the same EOA `0x70D78Ec75CF859d6FF337349489f32ed2DA1aa3E`, the DSM in transaction `0x7ffbc6a5a8e8db2be70bd8cd4ab8beb60e71f80e23549789bf1d7f3bf689d602` at block `25940562` and the locator implementation in `0x485bc670994189b18a93d47c6d14aabd5122b833866059ba8aa7cfb9f4ca8e0e` at block `25940593`, both plain `CREATE` with no proxy involved. The deployer holds no privilege over either contract.

Rebuilding from the audited commit with the project's own compiler settings reproduces both runtime bytecodes. The rebuilt code is identical to the deployed code. The verified sources published on the explorer, 8 files for the DSM and 2 for the locator, match the local repository byte-for-byte, including the OpenZeppelin 4.4 tree the DSM is pinned to.

The `DelegationFactory` was deployed by the EOA `0x60A4a17053A2D1b035CD83b9725B0081533C40b6` in transaction `0x96a42f88fd0109ae224640cf3d14046d7006f9aaf0391cfdccdd83bba9f713b3` at block `25875135`, also a plain `CREATE`. It takes no constructor arguments, holds no state and exposes no owner, admin or other privileged role, so the deployer retains nothing after deployment. Its runtime bytecode matches the rebuild from the audited commit exactly, with no immutables involved, and all verified source files match the local repository byte-for-byte, including the OpenZeppelin 5.4 tree the EDF is pinned to. The `DelegationContract` creation code embedded in the factory is byte-identical to the audited build, so every seat the factory produces is the audited implementation by construction, parameterised only by the `owner`, `delegate` and `cooldown` passed to `deploy()`.

All six contracts currently registered as DSM guardians appear among `DelegationContract`'s deployed through `DelegationFactory`.

The DSM constructor arguments decode to `_depositContract = 0x00000000219ab540356cBB839Cbe05303d7705Fa`, `_stakingRouter = 0xFdDf38947aFB03C621C71b06C9C70bce73f12999`, `_pauseIntentValidityPeriodBlocks = 6646` and `_maxOperatorsPerUnvetting = 200`, matching the canonical Beacon Chain deposit contract, the current mainnet StakingRouter, and the two parameter values carried over unchanged from the v4 DSM. Reading the deployed contract back confirms `VERSION() == 5`, an owner of `0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c` (the Lido Agent, same as the v4 DSM), a quorum of 4, and deposits unpaused.

The locator implementation was checked field by field against the one currently behind the locator proxy. Of the 24 addresses it stores, exactly one differs: `depositSecurityModule` moves from the v4 DSM

`0xF573E9E3de1f86B085417ab294f56E7920B4e9Be` to the new one. Every other component is byte-identical, so the implementation carries no unintended rewiring.

The six registered guardians are all genuine `DelegationContract` instances: each one's runtime code matches the audited EDF build, each advertises the ERC-1271 interface id, none is terminated, and none has a pending nomination. Every seat uses a cooldown of 172800 seconds (2 days), so the zero-cooldown case described in finding 6 does not occur in this deployment. All six delegate hot keys are plain EOAs with empty code, meaning none currently carries an EIP-7702 designator and the failure mode described in the Executive Summary is not realised at the time of verification. The six controlling entities are distinct Gnosis Safe 1.4.1 wallets with no modules and no transaction guard enabled; their thresholds are 3-of-5, 2-of-3, 2-of-5, 3-of-7, 2-of-3 and 2-of-4. Two of these sit below a 50% threshold ratio, which we note for completeness: a compromised owner key can only rotate its own seat's delegate, visibly and after the 2-day cooldown.

At the time of verification neither contract is enacted. The locator proxy `0xC1d0b3DE6792Bf6b4b37EccdcC24e45978Cfd2Eb` still points at the previous implementation, the proxy admin is the Lido Agent, and the proxy is not ossified. The new DSM does not yet hold `STAKING_MODULE_UNVETTING_ROLE` on the StakingRouter, which the v4 DSM still holds, so a DAO vote is still required. Consistent with the note in the Executive Summary, `getLastDepositBlock()` on the new DSM returns `25940562`, its own deploy block, rather than the block of the protocol's last actual deposit; `isMinDepositDistancePassed()` already returns true for all four staking modules, so this carries no practical delay.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

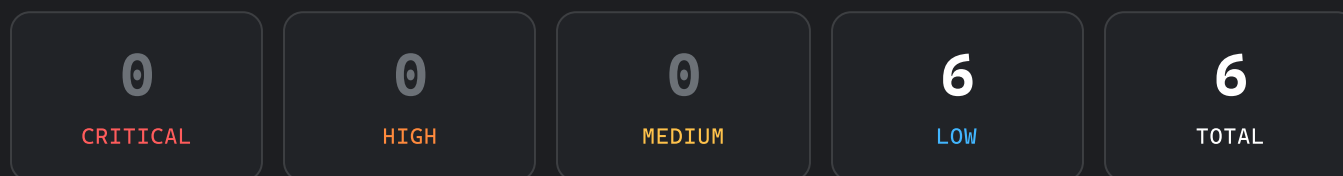
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	DepositSecurityModule guardian set is never reconciled with DelegationContract termination or revocation; effective quorum shrinks silently and dead contracts pass addGuardian	● LOW	● ACKNOWLEDGED
L-2	DelegationContract.nominateDelegate(): contract delegates allow silent re-delegation of a seat to another controlling entity and self-delegation loops	● LOW	● ACKNOWLEDGED
L-3	DepositSecurityModule._addGuardian() / _isValidGuardianSignature(): any ERC-165-advertising account can be a guardian, and the ECDSA acceptance path LIP-37 says was removed is still compiled in	● LOW	● ACKNOWLEDGED
L-4	The accepted guardian signature length is defined only by a disagreement between two OpenZeppelin versions, is undocumented, and is not pinned by any test	● LOW	● ACKNOWLEDGED
L-5	depositBufferedEther and pauseDeposits docstrings state "each component taking 32 bytes", but the new guardian field is packed as 20 bytes. A signer following the docs produces invalid signatures	● LOW	● ACKNOWLEDGED
L-6	No admission path into the protocol checks a delegation contract's cooldown, and the contract does not bound it, so a zero-cooldown guardian or oracle member can be onboarded at any time	● LOW	● ACKNOWLEDGED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 DepositSecurityModule guardian set is never reconciled with DelegationContract termination or revocation; effective quorum shrinks silently and dead contracts pass addGuardian

● LOW

● ACKNOWLEDGED

DESCRIPTION

A terminated or revoked DelegationContract keeps answering `supportsInterface` truthfully and `isValidSignature` with `0xffffffff`. DSM keeps it in `guardians`, keeps counting it toward the total, and `_addGuardian` accepts an already-terminated contract (`isTerminated()` is not consulted). Two irreversibly terminated seats turn a 4-of-6 committee into 4-of-4 with no on-chain signal. Pre-EDF, an offline guardian had the same effect, but it was recoverable by the operator; termination is not.

This issue is classified Low severity because the impact is deposit liveness for the duration of a governance cycle (Low impact) and it requires owner-side termination (Low likelihood).

RECOMMENDATION

We recommend rejecting `isTerminated()` contracts in `_addGuardian` when the guardian reports the `IDelegationContract` interface id, and adding `Terminated` / `DelegateRevoked` events of registered guardians to the monitoring set from LIP-37 migration step 4 with an explicit runbook for seat replacement.

CLIENT'S COMMENTARY

This is an operational verification task and a task for the vote tests, we deliberately did not add many checks to the contract.

L-2 `DelegationContract.nominateDelegate()`: contract delegates allow silent re-delegation of a seat to another controlling entity and self-delegation loops

● LOW

● ACKNOWLEDGED

DESCRIPTION

`nominateDelegate` rejects only `address(0)`, `OWNER`, the current and the pending delegate. Because OZ 5.4.0 routes any delegate with code to ERC-1271, an owner can nominate another `DelegationContract` (or any smart wallet) as its delegate. LIP-37's owner-immutability rationale ("prevents silent transfer of committee participation between organizations") is therefore not enforced.

The same missing check allows `nominateDelegate(address(this))`. Verification then recurses until the call-depth cap, burns gas proportional to what the caller forwarded, and returns invalid, so the seat is dead and every batch containing it is a gas sink for the relayer.

This issue is classified Low severity because the impact is accountability loss and seat-level liveness (Low impact) and only the seat's own owner can trigger it (Low likelihood).

RECOMMENDATION

We recommend reverting on `delegate == address(this)` and on delegates that report the `IDelegationContract` ERC-165 interface id, documenting in the Key Handling Policy that DSM and oracle delegates must be EOAs, and alerting in monitoring whenever `getDelegate().code.length != 0` for a registered seat.

CLIENT'S COMMENTARY

If a guardian is compromised, it can

- pause deposits
- unvet all keys in the protocol that have not been deposited

These are very serious actions in themselves, so against the background of these simple attacks (one only needs to call the DSM contract methods) more complex kinds of attacks do not seem worthwhile, since they add nothing new.

**L-3 DepositSecurityModule._addGuardian() /
_isValidGuardianSignature(): any ERC-165-advertising account
can be a guardian, and the ECDSA acceptance path LIP-37 says
was removed is still compiled in**

● LOW

● ACKNOWLEDGED

DESCRIPTION

LIP-37 states that "every guardian is a DelegationContract - EOA guardians are no longer supported" and that the redeployed DSM "does not retain the legacy ECDSA-against-an-EOA verification", and specifies a direct `IERC1271(guardian).isValidSignature(...) == MAGIC` call. The implementation differs on both points:

- `_addGuardian` gates only on `ERC165Checker.supportsInterface(addr, 0x1626ba7e)`: any contract, a Safe with a suitable handler, or an EIP-7702 EOA whose designated code advertises ERC-165/ERC-1271 passes. `isTerminated()` is never checked.
- `_isValidGuardianSignature` uses OZ 4.4.1 `SignatureChecker.isValidSignatureNow`, which runs `ECDSA.tryRecover` first and returns true when the recovered address equals the guardian, before ever calling `isValidSignature`.

Consequently an EIP-7702 EOA registered as guardian is authorised by its own raw key regardless of what its delegated code says: a designator pointing at code that rejects every signature, or at a real DelegationContract's runtime with a plausible `owner()`, still lets the EOA key pass `depositBufferedEther` and `pauseDeposits`. Such a seat has no EDF revocation or termination, only DAO removal. Registration is DAO-only, so this is a spec mismatch rather than an attack.

This issue is classified Low severity because the impact is the loss of EDF guarantees for one seat (Low impact) and only the DAO can register such a guardian (Low likelihood).

RECOMMENDATION

We recommend replacing the `SignatureChecker` call with the direct ERC-1271 call that LIP-37 specifies, so that no ECDSA path exists in the DSM, and, in `_addGuardian`, rejecting accounts whose code is a 7702 designator and requiring the DelegationContract `!isTerminated()`, then aligning the LIP text with whichever policy is chosen.

CLIENT'S COMMENTARY

This is an operational verification task, we ask guardians to provide the addresses of the deployed contracts in advance, and we also check that the contract is the correct one. Additional checks do not add any protection for us: if a guardian is compromised or wants to harm the protocol, it is easier for them to issue a valid contract and then pause the protocol.

L-4 The accepted guardian signature length is defined only by a disagreement between two OpenZeppelin versions, is undocumented, and is not pinned by any test

● LOW

● ACKNOWLEDGED

DESCRIPTION

This issue has been identified in the signature verification path of `DepositSecurityModule` v5 (`contracts/0.8.9/DepositSecurityModule.sol:525-531` , `_isValidGuardianSignature`) in combination with the `DelegationContract` it delegates to. Verification is two-tiered, and the two tiers accept different signature lengths:

- DSM (OpenZeppelin 4.4): `SignatureChecker.isValidSignatureNow` first runs `ECDSA.tryRecover(hash, bytes)` , which accepts both 65-byte `r|s|v` and 64-byte EIP-2098 `r|vs` signatures. The recovered address is the hot key, never the `DelegationContract` , so this branch is unreachable for every production guardian (`_addGuardian` requires an ERC-1271 contract, `:320-322`) and the check falls through to the ERC-1271 `staticcall` .
- `DelegationContract` (OpenZeppelin 5.4, `execution-delegation-framework/src/DelegationContract.sol:130-140`): `ECDSA.tryRecover(bytes32, bytes)` accepts 65 bytes only. 64 bytes yields `InvalidSignatureLength` and `isValidSignature` returns `0xffffffff` .

The effective rule - "guardian signatures must be exactly 65 bytes with `v` value 27, 28 - is therefore not stated anywhere in DSM v5: it is the intersection of two library behaviours, one of them living in a different repository pinned to a different OpenZeppelin version. Nothing in the DSM code, its docstrings, or its test suite records it. A 64-byte signature surfaces as the generic `InvalidSignature()` revert, indistinguishable from a wrong signer or a stale message.

The issue is classified as Low severity because no funds are at risk and rejection fails closed.

RECOMMENDATION

We recommend stating the accepted format explicitly in the docstrings of `depositBufferedEther` , `pauseDeposits` and `unvetSigningKeys` . Adding a unit test that asserts a 64-byte signature is rejected, executed against a real `DelegationContract` compiled from the pinned EDF commit. Optionally normalising 64-byte input to 65 bytes in `_isValidGuardianSignature` (or rejecting any length other than 65 up front with a dedicated error) so the DSM's contract with its callers no longer depends on which OpenZeppelin version the guardian contract happens to use.

CLIENT'S COMMENTARY

This is covered by tests in the council and in the depositor, we know that one version of OpenZeppelin supports 64 and 65 bytes, and the other only 65 (in EDF), and we took this into account. Additionally, we decided not to write a comment about this in the contracts.

L-5 depositBufferedEther and pauseDeposits docstrings state "each component taking 32 bytes", but the new guardian field is packed as 20 bytes. A signer following the docs produces invalid signatures

● LOW

● ACKNOWLEDGED

DESCRIPTION

This issue has been identified in the message-format documentation of `DepositSecurityModule` v5. The v5 change inserts the guardian address into every signed message to bind a delegate signature to one `DelegationContract`:

```
keccak256(abi.encodePacked(
  ATTEST_MESSAGE_PREFIX, guardian, blockNumber, blockHash,
  depositRoot, stakingModuleId, nonce))
```

`abi.encodePacked` encodes an `address` as 20 bytes, not 32. The docstrings were carried over from v4 and still read "each component taking 32 bytes" ([contracts/0.8.9/DepositSecurityModule.sol:367](#) for `pauseDeposits`, :456 for `depositBufferedEther`). A bot implemented from the docstring (left-padding the guardian to 32 bytes) will sign a different hash than the contract computes and every signature will be rejected with `InvalidSignature()`.

The issue is classified as Low severity because it does not affect on-chain safety and the in-repo tooling is consistent with the code.

RECOMMENDATION

We recommend correcting both docstrings to state that `guardian` is packed as 20 bytes (or to give the exact byte layout), and adding a reference test vector (message fields, expected hash, expected 65-byte signature) that bot implementers can check against.

CLIENT'S COMMENTARY

Valid, but we do not think that an incorrect comment is a Low, it is rather an Info.

The misleading text does not affect anything in the end, we decided not to touch it, we will create an issue.

L-6 No admission path into the protocol checks a delegation contract's cooldown, and the contract does not bound it, so a zero-cooldown guardian or oracle member can be onboarded at any time

● LOW

● ACKNOWLEDGED

DESCRIPTION

The cooldown is what makes a compromised owner's nomination visible before it takes effect. with `cooldown == 0` a stolen owner key swaps the delegate in one transaction and EDF adds nothing over v4 for that controlling entity. The value is chosen by the controlling entity at deployment and is immutable, so the only place to reject a bad one is at admission.

The issue is classified as Low severity because a zero cooldown weakens only the controlling entity that chose it and the DAO can decline to onboard such a contract. The residual risk is that nothing codifies the rule and the reference tooling defaults to zero.

RECOMMENDATION

We recommend defining a minimum cooldown in the guardian/oracle onboarding policy and enforcing it in code.

CLIENT'S COMMENTARY

This is also resolved operationally and at the level of the vote tests, we check the parameters with which we add a guardian.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(71437d9eb293cebe1ebd56e94679f5af_img.jpg\)](#) [!\[\]\(03fdfa7eaf8f56ef9d3e052966eac184_img.jpg\)](#) [!\[\]\(785cf91d72a120426323d1ce3ca4d2d3_img.jpg\)](#) [!\[\]\(9cb67ed3feecfd41d989096ea97806c2_img.jpg\)](#)